

EE1102 : Introduction to Programming
Aug-Nov 2023 - Assignment 4

Things to Know for this Lab

All writing and reading requires including `stdio.h`:

```
#include<stdio.h> /* this is a comment line (can be on separate line too) */
```

Basic structure of a C program (call it `foo.c`):

```
#include....
int main(){ /* start of the main function. All execution starts here */
    .... /* code */
    return (...) /* return value to the operating system */
} /* end of the main program */
```

To compile the code: `gcc foo.c -o foo` compiles the program `foo.c` and creates the executable `foo`.

Most programs we write will also require `math.h`, which is the include file for the `libmath.so` library. The 'so' indicates that the library is a "shared" library, which means that a single copy of the library function is in the memory which is used by all programs that require it.

To compile such a program, we have to teach the compiler that we need `libmath.so`. For that we execute: `gcc foo.c -lm -o foo`. (Actually the current gcc on my computer seems to use `libmath.so` automatically. So the `-lm` may not be required.)

Printf:

To write something, use `printf("...")`.

```
printf("This is a line\n");
This is a line
```

To print multi-line output

```
printf("This is\na line\n"); /* each \n starts a new line */
This is
```

a line

Note: if you had written

```
printf("This is\n a line\n");
```

A space would have been added ahead of the 'a' in the second line.

Integer variables and assignment and printing the same:

To declare an integer variable and assign it a value

```
int a;  
a=4;
```

Can also assign the value in the same line:

```
int a=4;
```

To write something that includes an integer variable (printf conversion specification for integer is %d)

```
printf("The value of a = %d\n",a); /* output printed, with %d replaced by value */  
The value of a = 4
```

Note that the following is allowed: %6d - print as integer, using 6 spaces

```
int a=8, b=120;  
printf("The value of a = %3d and b = %3d\n",8,15); /* use 2 digits for all output */  
The value of a = 8 and b = 12 /* note extra space for 8 */
```

Addition of variables and constants

```
int b=10;  
int c=15; /* can give the variable a value inside the declaration itself */  
int d,e;  
d=a+b+c; /* addition of variables */  
e=a+4+c; /* addition of variables and constants (here 4 is a constant) */
```

String constants, declaration and output.

To declare a string variable, and assign it a value

```
char name[ ] = "Abc Xyz";
```

The syntax will be explained later, but this is a simple way to declare string constants. To print some strings, use

```
printf("My name is %s, and my home is in %s\n", name, city);
```

%s is the symbol used to indicate a string. It will be taken from the list of variables following the format string. Again, we can modify the output of the string:

```
printf("My name is %10s, and my home is in %10s\n", name, city);
```

This limits the number of spaces to be used for name and city to be a maximum of 10. If the string has 10 spaces or less, it will print out correctly with additional spaces as required. But if the string is larger, C will print out the whole string so that the output is readable.

Float and Double variables, assignment and output:

To declare a real number, and assign it a value

```
float a = 4.5;
```

This creates a real number that uses 32 bits of storage. The mantissa is 24 bits in size including sign, and the exponent is 8 bits in size. Note that exponent cannot exceed 7 bits as there is an offset. To print float numbers use

```
printf("I am %f cm tall\n", 66.5);
```

Note that the value for height can also be taken from a constant

Long long int variables:

To declare a long int variable. This is a little strange. "int" is not a fixed size. It depends on the cpu of the computer. So a long integer variable is called "long long int"

```
long long int aa = 123456789;  
printf("aa = %lld\n", aa);
```

This creates an integer that uses 64 bits of storage. Such an integer can store much larger size integer values. For example,

```
long long int amnt=1000; /* amount in crores to be printed out */  
Int crore = 10000000; /* Number of Rs in 1 crore */  
amnt = amnt*crore; /* amnt converted to Rs. */  
printf("1000 crores is %lld Rs.", amt );
```

To declare a long real number, and assign it a value

```
double b = 3.5e250
```

This creates large real numbers whose mantissa or exponent will not fit into a float variable.

Scanf:

If we want the user to enter some information, we use the scanf function. The format is

```
scanf("format",&arg1, &arg2, ..., &argn);
```

For example, (assuming #include<stdio.h> is present at the top)

```
Int a,b;  
printf("enter a and b: "); /* help message */  
scanf("%d %d", &a, &b);  
print("a = %d, b = %d \n",a,b);
```

The '&' symbol is needed to give scanf not the variable, but the memory location where the variable exists. Printf takes the value present at a memory. So the variable name is all that is needed. But here, we need the memory location so that scanf can write the value entered by the user into that memory.

The remainder function:

We have already looked at +, -, * and /. An additional operator is the modulus operator. $x\%y$ is defined to be the remainder when x is divided by y . Both / and % are machine dependent when the arguments are negative.

Logical operators:

Relational operators are operators that act on logical expressions to get logical answers.

If $i=4$ and $j=7$, then

$i < j$ is True

$2*i < j$ is false.

$(i < j) \ \&\& \ (2*i > j)$ is True as both statements are true

These will be very important once we talk about control statements.

Control Statements: if, for and while and switch:

```
if( logical expression ){  
    Code to be executed if TRUE  
}
```

```
if( logical expression ){  
    Code to be executed if TRUE  
}else{  
    Code to be executed if FALSE  
}
```

```
if( logical expression 1 ){  
    Code to be executed if logical expression 1 is TRUE  
}else if( logical expression 2 ){  
    Code to be executed if logical expression 2 is TRUE  
}else{  
    Code to be executed if logical expression 1 is FALSE  
}
```

IF statements can be nested. Eg from previous assignment:

```
if( D >= 0.0 ){  
    if( D == 0.0 ){/* nested if */  
        printf("The roots are identical \n");  
        ... code to find the roots' values  
    } else { /* this else goes with nested if */  
        printf("The roots are real and distinct\n");  
        ... code to find the roots' values  
    }  
} else { /* this goes with outer if statement */  
    printf("The roots are complex and distinct\n");  
    ... code to find the roots  
}
```

While and for:

```
while( logical expression ){ /* starts loop */  
    Code                      /* must modify logical expression to terminate the loop */  
}
```

```
for( expr 1 ; logical expr 2 ; expr 3 ){  
    Code                      /* executed till expr2 becomes false */  
}
```

For is the same as the following code:

```
Expr 1;  
while( logical expr 2 ){  
    Code;  
    Expr 3;  
}
```

Example: print all the digits in order:

```
printf("digits: \n");  
for( int i=0 ; i<10 ; i++ ){  
    printf("%1d\n",i);  
}
```

New additions to this section:

More Control structures:

Conditional Expression: (p 51-52 of K&R)

`(expr1)? Expr 2 : expr 3;`

Eg. `max(a,b)` is implemented in C as

`(a>b) ? a : b;`

Eg. printing 10 lines of an array per page:

`for( i=0 ; i<n ; i++ )`

`printf(“%6d%c”, a[i], (i%10==9 || i==n-1 ) ? \n : “ ”);`

The `%c` prints blanks except when `i%10` is 9 or `i` is `n-1`. In those cases inserts a `\n` (newpage)

Do While:

`do{`

Code

`}while( logical expression )`

It is like `while( expr ){`

Code

`}`

except that the Code is executed at least once.

Eg. To extract the digits of a positive number s: p64 of K&R

```
Do{  
    S[i++] = n%10 + '0'    /* get next remainder and convert into ascii */  
}while( (n /= 10) > 0 );    /* divide n by 10, and loop if >0 */
```

If s is less than 10, the division yields zero and terminates too soon. So the do ... while.

Break and Continue

If we want to exit the loop you are in prior to the exit condition, use BREAK. To merely skip the rest of the body of the loop and go to the next iteration, use CONTINUE.

Eg: remove trailing blanks, tabs and newlines. Line is in 's'.

```
for( n = strlen(s)-1 ; n>=0 ; n-- ){    /* count down n from length of string */  
    if( s[n] != ' ' && s[n] != '\t' && s[n] != '\n' )  
        break;    /* exit the for loop if we have only blanks, tabs or \n */  
}    /* indicates the loop that is skipped.*/
```

Eg: skip processing certain elements in a loop (p. 65 of K&R)

```
for( i=0 ; i<n ; i++ ){    /* loop to process elements of an array */  
    if( a[i]<0 )  
        continue;    /* do not process this element as it is -ve */  
    ....    /* process remaining elements */  
}
```

Assignment

Problem 1. Write a C program called `ROLLNO_prime-number.c` that reads an integer n from the *stdin* and checks if n ($n \geq 2$) is a prime number. Check that n is not negative or less than 2. You should use a *while* loop for this problem.

Problem 2. Write a C program called `ROLLNO_binomial_coefficient.c` to input two positive integers (N and K) and compute the binomial coefficient N choose K , $\binom{N}{k} = \frac{N!}{(N-k)!k!}$.

You should use a *for* loop for this problem. You should check

if your program works for cases such as $\binom{100}{98}$. If not, you need to find a way to compute it.

Note that $N!/(N-k)!k!$ is $(k+1) \dots (N)/(1*2* \dots (N-k))$ which can eliminate many overflow situations.

Problem 3. Write a C program called `ROLLNO_sum_of_series.c` that takes as input a positive integer n and prints the value of the following sum:

$$1 - (1/2) + (2/3) - + \dots - + ((n-1)/n)$$

Does the series converge? Do the even terms and the odd terms separately converge? Print the sums for even n values from 2 upto $n=20$, and separately for odd n values, from 3 till $n=21$, like the following:

N	even sum (N-1)	odd sum (N)
3	<Answer>	<Answer>
...		

The answers should be correct to 4 digits after the decimal point.